

■ OpenShift

Guide des actions sur une base de données hébergée

Port-Forwarding • Connexion • Sauvegarde • Restauration • Monitoring & plus

Ce tutoriel présente, sous forme de tableaux de référence rapide, les principales actions réalisables sur une base de données (PostgreSQL, MySQL, MongoDB...) déployée dans un cluster **OpenShift**. Chaque section couvre : la commande ou la procédure, les prérequis, les options clés et des exemples concrets.

1. Port-Forwarding — Accès local à la base de données

Action	Commande / Procédure	Options clés	Exemple concret
Forwarder un port PostgreSQL	<code>oc port-forward <pod> <local>:<remote></code>	• Port local libre • Nom exact du pod • Namespace (-n)	<code>oc port-forward pod/pg-0 5432:5432 -n myapp</code>
Forwarder via un Service	<code>oc port-forward svc/<service> <local>:<remote></code>	• Utilise le sélecteur du Service • Plus stable qu'un pod direct	<code>oc port-forward svc/postgres 5432:5432 -n myapp</code>
Forwarder MySQL	<code>oc port-forward svc/mysql 3306:3306</code>	• Lancer en arrière-plan : & (Linux/Mac) • Tuer avec <code>kill %1</code>	<code>oc port-forward svc/mysql 3306:3306 -n myapp &</code>
Forwarder MongoDB	<code>oc port-forward svc/mongodb 27017:27017</code>	• Vérifier que le port n'est pas déjà occupé localement	<code>oc port-forward svc/mongodb 27017:27017 -n myapp</code>
Connexion client après forward	<code>psql -h 127.0.0.1 -p 5432 -U user -d db</code>	• Le forward doit rester actif • Utiliser 127.0.0.1 (pas localhost)	<code>psql -h 127.0.0.1 -p 5432 -U admin -d mydb</code>

2. Connexion et accès à la base de données

Action	Commande	Prérequis	Remarques
Ouvrir un shell dans le pod DB	<code>oc exec -it <pod> -- bash</code>	Droits exec sur le pod	<code>oc exec -it pod/pg-0 -n myapp -- bash</code>
Connexion psql directe	<code>oc exec -it <pod> -- psql -U user -d db</code>	PostgreSQL installé dans le pod	<code>oc exec -it pod/pg-0 -- psql -U admin -d mydb</code>
Connexion mysql directe	<code>oc exec -it <pod> -- mysql -u user -p db</code>	MySQL client dans le pod	<code>oc exec -it pod/mysql-0 -- mysql -u root -p mydb</code>
Lire les secrets de connexion	<code>oc get secret <name> -o jsonpath='{.data.*}' base64 -d</code>	Droits sur les Secrets	<code>oc get secret pg-secret -o jsonpath='{.data.password}' base64 -d</code>
Récupérer l'URL de la DB	<code>oc get svc <service> -o yaml</code>	Droits get/list sur Services	<code>oc get svc postgres -n myapp -o yaml grep clusterIP</code>

3. Sauvegarde (Backup) et Restauration

Action	Commande	Format / Options	Exemple
pg_dump — dump logique	oc exec <pod> -- pg_dump -U user db > dump.sql	• -F c : format custom • -F t : tar • --no-owner	oc exec pod/pg-0 -- pg_dump -U admin mydb -F c > mydb.dump
pg_dumpall — tous les DB	oc exec <pod> -- pg_dumpall -U postgres > all.sql	Inclut rôles et tablespaces	oc exec pod/pg-0 -- pg_dumpall -U postgres > all.sql
mysqldump	oc exec <pod> -- mysqldump -u user -p db > dump.sql	• --all-databases • --single-transaction (InnoDB)	oc exec pod/mysql-0 -- mysqldump -u root -pSecret mydb > mydb.sql
mongodump	oc exec <pod> -- mongodump --out /tmp/dump	• --db pour une seule DB • --gzip	oc exec pod/mongo-0 -- mongodump --db mydb --out /tmp/dump
Copier le dump localement	oc cp <pod>:/chemin/dump.sql ./dump.sql	• Sens inverse pour envoyer • Syntaxe : pod:/path local	oc cp pod/pg-0:/tmp/mydb.dump ./mydb.dump -n myapp
Restaurer PostgreSQL	oc exec -i <pod> -- psql -U user -d db < dump.sql	pg_restore pour format custom	oc exec -i pod/pg-0 -- pg_restore -U admin -d mydb -F c < mydb.dump
Restaurer MySQL	oc exec -i <pod> -- mysql -u user -p db < dump.sql	La DB cible doit exister	oc exec -i pod/mysql-0 -- mysql -u root -pSecret mydb < mydb.sql

4. Gestion des Pods et des Ressources

Action	Commande	Options / Flags	Exemple
Lister les pods DB	oc get pods -n <ns> -l app=<label>	-o wide pour IP/node -w pour surveiller	oc get pods -n myapp -l app=postgres -o wide
Redémarrer un pod	oc delete pod <pod> -n <ns>	Le ReplicaSet recrée le pod	oc delete pod pg-0 -n myapp
Scaler le déploiement	oc scale deployment <name> --replicas=N	0 replica = arrêt propre	oc scale deployment postgres --replicas=0 -n myapp
Voir les logs DB	oc logs <pod> -f --tail=100	-f = suivi temps réel --previous pour crashés	oc logs pod/pg-0 -f --tail=200 -n myapp
Décrire le pod	oc describe pod <pod> -n <ns>	Affiche events, limites, volumes	oc describe pod pg-0 -n myapp
Voir l'utilisation CPU/RAM	oc adm top pods -n <ns>	Metrics Server requis	oc adm top pods -n myapp --sort-by=memory

5. Gestion des Secrets et ConfigMaps

Action	Commande	Remarques	Exemple
Créer un Secret DB	<code>oc create secret generic <name> --from-literal=key=val</code>	Encodage base64 automatique	<code>oc create secret generic pg-creds --from-literal=password=S3cr3t -n myapp</code>
Voir les Secrets	<code>oc get secrets -n <ns></code>	-o yaml pour voir les données	<code>oc get secret pg-creds -o yaml -n myapp</code>
Décoder un Secret	<code>oc extract secret/<name> --to=- 2>/dev/null</code>	Affiche les valeurs en clair	<code>oc extract secret/pg-creds --to=- -n myapp</code>
Modifier un Secret	<code>oc patch secret <name> -p '{"data":{"key":"b64val"}}'</code>	Valeur doit être en base64	<code>oc patch secret pg-creds -p '{"data":{"password":"UzNjcjN0"}}'</code>
ConfigMap — config DB	<code>oc create configmap <name> --from-file=postgresql.conf</code>	Monter via volumeMount	<code>oc create configmap pg-config --from-file=postgresql.conf -n myapp</code>

6. Volumes et Stockage Persistant (PVC)

Action	Commande / Ressource	Points clés	Exemple
Lister les PVC	<code>oc get pvc -n <ns></code>	Statut Bound = OK Pending = problème de stockage	<code>oc get pvc -n myapp -o wide</code>
Inspecter un PVC	<code>oc describe pvc <name> -n <ns></code>	Affiche StorageClass, capacité, accessModes	<code>oc describe pvc postgres-data -n myapp</code>
Agrandir un PVC	<code>oc patch pvc <name> -p '{"spec":{"resources":{"requests":{"storage":"10Gi"}}}}'</code>	StorageClass doit autoriser l'expansion	<code>oc patch pvc postgres-data -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}' -n myapp</code>
Copier des données vers un PVC	<code>oc cp ./data <pod>:/var/lib/pgsql/data/</code>	Pod doit être en cours d'exécution	<code>oc cp ./backup/ pod/pg-0:/var/lib/pgsql/ -n myapp</code>
Snapshot d'un PVC (VolumeSnapshot)	<code>oc apply -f snapshot.yaml</code>	Requiert VolumeSnapshotClass APIGroup: snapshot.storage.k8s.io	<code>kind: VolumeSnapshot name: pg-snapshot-01 spec.source.persistentVolumeClaimName: postgres-data</code>

7. Monitoring et Diagnostic

Action	Commande / Outil	Ce qu'on observe	Exemple
Événements du namespace	<code>oc get events -n <ns> --sort-by=.lastTimestamp</code>	Crashes, OOMKill, mount errors	<code>oc get events -n myapp --sort-by=.lastTimestamp tail -20</code>
Logs en temps réel	<code>oc logs -f <pod> -n <ns></code>	Erreurs SQL, connexions refusées	<code>oc logs -f pod/pg-0 -n myapp grep ERROR</code>
Métriques Prometheus	<code>oc get servicemonitor -n <ns></code>	Nécessite l'opérateur monitoring	<code>oc get servicemonitor postgres-monitor -n myapp -o yaml</code>
Vérifier la connectivité réseau	<code>oc exec <pod> -- nc -zv <svc> <port></code>	Teste l'accès depuis un pod vers la DB	<code>oc exec pod/app-0 -- nc -zv postgres-svc 5432</code>
Lister les connexions actives (PG)	<code>SELECT * FROM pg_stat_activity;</code>	Exécuter dans un exec psql	<code>oc exec pod/pg-0 -- psql -U admin -c 'SELECT pid,username,state FROM pg_stat_activity;'</code>
Tableau de bord OpenShift	Console Web > Workloads > Pods > pod > Metrics	CPU, RAM, réseau en graphique	Naviguer : Observe > Dashboards > Kubernetes / Compute Resources

8. Opérations avancées

Opération	Commande / Approche	Prérequis	Exemple / Note
Migration de données inter-namespaces	<code>pg_dump source oc exec -i dest -- psql ...</code>	Accès aux deux namespaces	<code>pg_dump -h 127.0.0.1 mydb oc exec -i pod/pg-new -- psql -U admin newdb</code>
Mise à jour de l'image DB	<code>oc set image deployment/<name> <container>=<image:tag></code>	Tester en staging avant prod	<code>oc set image deployment/postgres postgres=postgres:15 -n myapp</code>
Job de backup automatique	CronJob OpenShift avec pg_dump	ServiceAccount avec droits exec	<code>kind: CronJob schedule: "0 2 * * *" → lance pg_dump vers un PVC ou S3</code>
Activation TLS sur la DB	Monter cert + clé via Secret, configurer postgresql.conf	Certificats valides (cert-manager)	<code>ssl=on / ssl_cert_file=/certs/tls.crt</code> dans postgresql.conf
Haute disponibilité (Patroni)	Opérateur Crunchy Data / Patroni via StatefulSet	Opérateur PostgreSQL installé	<code>oc get postgresql -n myapp oc describe postgresql hippo</code>
Accès externe via Route	<code>oc create route passthrough --service=<svc> --port=5432</code>	TLS requis pour passthrough Port non-HTTP (SNI)	<code>oc create route passthrough pg-route --service=postgres --port=5432 -n myapp</code>

Prérequis généraux : oc CLI connecté au cluster (oc login), droits RBAC suffisants sur le namespace cible. **Remplacer** <pod>, <ns>, <name> par les valeurs réelles de votre environnement.